Платформено-независими програмни езици

Лекция 1

Въведение в Java и основи на езика

Платформата Java, първа програма, типове, оператори, управляващи конструкции, масиви и низове

проф. д-р Даниела Гоцева
Учебна 2026/2027 г.



Съдържание на лекцията

1. Курсът и платформата Java
2. Първа програма и инструменти
3. Типове, променливи и оператори
4. Управляващи конструкции
5. Методи и масиви
6. Низове и изброими типове

01

Курсът и платформата Java

Цели, съдържание, литература и екосистема

Цел на курса

- Да усвоим **добрите програмистки практики** и да придобием трайни умения
- Това не е само курс по Java, а по **философията на ООП**
- Ще видим как принципите на ООП се прилагат на практика в съвременна Java
- Ще пишем код, който е **четим, тестван и преизползваем**
- Ще използваме инструментите, с които се работи в индустрията днес
- Ще се научим да използваме ИИ като помощник, без да губим контрол над кода

Съдържание на курса

№	Тема	№	Тема
1	Въведение и основи на езика	6	Stream API и функционален стил
2	Класове и обекти	7	Изключения и вход/изход
3	Наследяване и полиморфизъм	8	Нишки и мрежово програмиране с TCP
4	Интерфейси, вложени класове, ламбда изрази	9	Тестване и логване
5	Generics и колекции	10	Java и изкуствен интелект

Литература и ресурси

- C. Horstmann, **Core Java, Vol. I – Fundamentals**, 13th ed. (Java 21)
- J. Bloch, **Effective Java**, 3rd ed.
- B. Eckel, **Thinking in Java** – класика за философията на ООП
- Официални уроци: `docs.oracle.com/javase/tutorial`
- Документация на API: `docs.oracle.com/en/java/javase/21`
- Материали на курса: `dgotseva.com`

От Python към Java: основни разлики

	Python	Java
Типизация	динамична: `x = 5`, после `x = "a"`	статична: `int x = 5;` – типът е фиксиран
Блокове	отстъп (indentation)	`{ }`, отстъпът е само за четимост
Край на оператор	нов ред	`;`
Изпълнение	интерпретатор	компилатор `javac` → байткод → JVM
Входна точка	кодът на модула	метод `main` в клас
Структура	функции и класове в модул	всичко е в клас
Грешки в типовете	при изпълнение	при компилиране

Java – кратка история

- 1995 г. – Sun Microsystems, екип на **Джеймс Гослинг**
- Девиз: **„Write once, run anywhere“**
- 2004 г. – Java 5: generics, enum, анотации
- 2010 г. – Oracle придобива Sun
- 2014 г. – Java 8: ламбда изрази и Stream API
- От 2017 г. – нова версия на **всеки 6 месеца**
- LTS версии: 8, 11, 17, **21** (2023), **25** (2025)

Съвременна Java: какво е ново?

Версия	Ключови нововъведения
Java 10–11	`var`, нов `HttpClient`, стартиране на файл без компилация
Java 14–17	`switch` изрази, text blocks, `record`, `sealed` класове
Java 21 LTS	pattern matching за `switch`, record patterns, виртуални нишки
Java 22–25	безименни променливи `_`, `void main()`, гъвкави конструктори

Как работи Java: компилиране и изпълнение

- Изходният код (`.java`) се компилира от `javac` до **байткод** (`.class`)
- Байткодът не е машинен код – той е за **виртуалната машина (JVM)**
- JVM интерпретира байткода и с **JIT компилатор** превежда „горещите“ части в машинен код
- Един и същ `.class` файл работи под Windows, macOS и Linux
- Това е смисълът на **платформената независимост**

JDK, JRE и JVM

JDK – за разработчика

- компилатор `javac`
- стартер `java`, `jshell`
- `javadoc`, `jar`, `jdeps`
- съдържа и средата за изпълнение

JVM – за изпълнението

- зарежда и проверява класовете
- изпълнява байткода, JIT
- управлява паметта (garbage collector)
- изпълнява нишките

02

Първа програма и инструменти

Hello World, компилиране, JShell и среда за разработка

Hello, World!

- Всеки код в Java е в **клас**
- Името на публичния клас = името на файла
- `main` е входната точка на програмата
- `System.out.println` отпечатва ред на конзолата

```java
// Файл: HelloWorld.java
public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Компилиране и стартиране от терминала

```
$ javac HelloWorld.java        # създава HelloWorld.class
$ java HelloWorld              # стартира JVM с класа
Hello, World!

$ java HelloWorld.java         # от Java 11: директно от изходния код
Hello, World!

$ java -version
openjdk version "21.0.5" 2024-10-15 LTS
```

Стартирането директно от `.java` е удобно за малки програми и за учене

Java 25: опростена входна точка

```java
// Файл: Hello.java  (Java 25+)
void main() {
    IO.println("Здравей, свят!");
    String name = IO.readln("Как се казваш? ");
    IO.println("Приятно ми е, " + name);
}
```

Без явен клас и без `static` – подходящо за първи стъпки. В реалните проекти пишем пълната форма.

JShell – интерактивна конзола

- Идва с JDK (от Java 9)
- Изпълнява изрази и оператори веднага
- Удобна за експерименти по време на лекцията
- `/exit` – изход, `/vars` – списък с променливи

```
$ jshell
|  Welcome to JShell -- Version 21

jshell> int x = 6 * 7
x ==> 42

jshell> "Java".repeat(3)
$2 ==> "JavaJavaJava"

jshell> Math.sqrt(2)
$3 ==> 1.4142135623730951
```

Среда за разработка: Eclipse IDE

- В курса използваме **Eclipse IDE for Java Developers** – безплатна, `eclipse.org`
- Проект: File → New → Java Project; клас: New → Class
- Стартиране: Run As → Java Application (Ctrl+F11)
- Какво ни дава IDE:
  - автоматично допълване (Ctrl+Space) и бързи поправки (Ctrl+1)
  - откриване на грешки докато пишем
  - дебъгер – точки на прекъсване и изпълнение стъпка по стъпка
  - рефакториране, интеграция с Git и Maven
- Алтернативи: IntelliJ IDEA, VS Code с Extension Pack for Java

Особености при писане на код

- Чувствителен към малки и главни букви: `Count` ≠ `count`
- Всеки оператор завършва с `;`
- Блоковете се ограждат с `{ }`
- Коментари: `//`, `/* */` и документиращи `/** */`

```java
/**
 * Документиращ коментар – за javadoc.
 */
public class Rules {
    // едноредов коментар
    public static void main(String[] args) {
        int count = 1;   /* блоков коментар */
        int Count = 2;   // друга променлива!
        System.out.println(count + Count);
    }
}
```

Конвенции за именуване

Елемент	Стил	Пример
Клас, интерфейс, record	PascalCase	`BankAccount`, `Shape`
Метод, променлива	camelCase	`getBalance()`, `totalSum`
Константа	UPPER_SNAKE_CASE	`MAX_SIZE`, `PI`
Пакет	малки букви, домейн обратно	`bg.tu.pik3.bank`

Пример: факториел

```
public class Factorial {

    static long factorial(int n) {
        long result = 1;
        for (int i = 2; i <= n; i++) {
            result *= i;
        }
        return result;
    }

    public static void main(String[] args) {
        for (int n = 0; n <= 10; n++) {
            System.out.printf("%2d! = %d%n", n, factorial(n));
        }
    }
}
```

Статичен метод с параметър, цикъл `for`, тип `long` за големи стойности и форматиран изход с `printf`

Вход от клавиатурата: Scanner

- `Scanner` чете от `System.in`
- `nextInt`, `nextDouble` – числа
- `nextLine` – цял ред
- Внимание: след `nextInt` остава знакът за нов ред

```java
import java.util.Scanner;

public class Greeting {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        System.out.print("Име: ");
        String name = in.nextLine();

        System.out.print("Година на раждане: ");
        int year = in.nextInt();

        System.out.printf("%s, ще навършите %d г.%n",
                name, 2027 - year);
    }
}
```

03

Типове, променливи и оператори

Примитивни и референтни типове, var, преобразувания

Примитивни типове

Тип	Размер	Диапазон / стойности	Пример
`byte`	8 бита	−128 … 127	`byte b = 10;`
`short`	16 бита	−32 768 … 32 767	`short s = 1000;`
`int`	32 бита	≈ ±2,1 милиарда	`int n = 42;`
`long`	64 бита	≈ ±9,2 × 10^18	`long big = 9_000_000_000L;`
`float`	32 бита	≈ 7 значещи цифри	`float f = 3.14f;`
`double`	64 бита	≈ 15 значещи цифри	`double d = 2.718;`
`char`	16 бита	Unicode символ	`char c = 'Я';`
`boolean`	–	`true` / `false`	`boolean ok = true;`

Примитивни и референтни типове

Примитивни

- 8 вградени типа
- променливата съдържа самата стойност
- не могат да бъдат `null`
- сравняване с `==`

Референтни

- класове, интерфейси, масиви, `String`, `record`
- променливата съдържа **референция** към обект
- могат да бъдат `null`
- сравняване на съдържание с `equals()`

Променливи и литерали

- Променливата се декларира с тип и име
- Трябва да е инициализирана преди употреба
- `_` разделя цифрите за четимост
- Префикси: `0x` (16-ична), `0b` (двоична)
- Суфикси: `L` за `long`, `f` за `float`

```
int age = 21;
double price = 19.99;
long population = 8_100_000_000L;
int mask = 0b1010_1010;
int color = 0xFF8800;
char letter = 'A';
char unicode = 'Ж';   // Ж
boolean passed = age >= 18;

final double VAT = 0.20;   // константа
// VAT = 0.25;  -> грешка при компилиране
```

Извеждане на типа с var

- От Java 10 – за **локални** променливи
- Типът се извежда от инициализатора
- Типът остава **статичен** – `var` не е динамична типизация
- Използвайте го, когато типът е очевиден

```
var count = 10;                 // int
var name = "Java";              // String
var list = new ArrayList<String>();

for (var item : list) {
    System.out.println(item);
}

// var x;           // грешка: няма инициализатор
// var y = null;    // грешка: типът е неизвестен
// count = "десет"; // грешка: count е int
```

Оператори

Група	Оператори
Аритметични	`+  -  *  /  %  ++  --`
Сравнение	`==  !=  <  >  <=  >=`
Логически	`&&  ||  !` (с кратко съединяване)
Побитови	`&	^ ~ << >> >>>`
Присвояване	`=  +=  -=  *=  /=  %=`
Условен	`условие ? стойност1 : стойност2`
Проверка на тип	`instanceof`

Целочислено деление и препълване

- `int / int` дава `int` – дробната част се отрязва
- `%` – остатък от деление
- Препълването **не** предизвиква грешка!
- `Math.addExact` хвърля изключение при препълване

```
System.out.println(7 / 2);      // 3
System.out.println(7 / 2.0);    // 3.5
System.out.println(-7 % 3);     // -1

int max = Integer.MAX_VALUE;
System.out.println(max + 1);    // -2147483648

System.out.println(0.1 + 0.2);  // 0.30000000000000004

// Math.addExact(max, 1);  -> ArithmeticException
```

Преобразуване на типове

- **Разширяващо** (автоматично): `int` → `long` → `double`
- **Стесняващо** (явно, с cast): може да загуби данни
- В изрази `byte`, `short`, `char` се повишават до `int`

```
int i = 100;
long l = i;              // автоматично
double d = l;            // автоматично

double pi = 3.99;
int whole = (int) pi;  // 3 — отрязва
long big = 300;
byte b = (byte) big;   // 44 — загуба!

byte x = 10, y = 20;
// byte z = x + y;     // грешка: резултатът е int
byte z = (byte) (x + y);
```

Класът Math

- Статични методи за математически функции
- Не се създава обект: `Math.sqrt(...)`
- `Math.round` закръгля до цяло число
- `Math.floorDiv`, `Math.floorMod` – деление с отрицателни числа

```
Math.sqrt(16);              // 4.0
Math.pow(2, 10);            // 1024.0
Math.abs(-7);               // 7
Math.max(3, 9);             // 9
Math.round(2.5);            // 3
Math.floor(2.7);            // 2.0
Math.ceil(2.1);             // 3.0
Math.hypot(3, 4);           // 5.0
double pi = Math.PI;        // 3.141592653589793
Math.floorMod(-7, 3);       // 2  (a -7 % 3 e -1)
```

Случайни числа

- `Math.random()` – число в [0, 1)
- `Random` – генератор с методи за различни типове
- `nextInt(a, b)` – цяло число в [a, b) (Java 17+)
- Със зададено начало (seed) – повторяема поредица

```java
double r = Math.random();

Random rnd = new Random();
int dice = rnd.nextInt(1, 7);      // 1..6
double g = rnd.nextGaussian();
boolean coin = rnd.nextBoolean();

Random fixed = new Random(42);     // seed
System.out.println(fixed.nextInt(100));
// при всяко пускане — едно и също число
```

Обхват на променливите

- Променливата е видима **от декларацията до края на блока**
- Не може да се декларира същото име във вложен блок
- Полетата на класа са видими във всички методи

```
public class Scope {
    static int counter = 0;      // поле

    public static void main(String[] a) {
        int x = 5;               // до края на main
        if (x > 0) {
            int y = x * 2;       // само в if
            counter += y;
            // int x = 1;        // грешка!
        }
        // print(y);             // грешка!
        System.out.println(counter);
    }
}
```

04

Управляващи конструкции

Условия, switch изрази и цикли

Условен оператор if / else

- Условието трябва да е от тип `boolean`
- `else if` за поредица от проверки
- Винаги използвайте `{ }` – дори за един ред

```
int score = 78;
String grade;

if (score >= 90) {
    grade = "Отличен";
} else if (score >= 75) {
    grade = "Много добър";
} else if (score >= 60) {
    grade = "Добър";
} else {
    grade = "Слаб";
}

String status = score >= 50 ? "взет" : "невзет";
```

switch: старият и новият стил

Класически (с двоеточие)

- `case X:` и `break`
- пропуснат `break` → „пропадане“
- чест източник на грешки
- само оператор, не връща стойност

Модерен (със стрелка)

- `case X ->` без `break`
- няколко етикета: `case 1, 2 ->`
- може да е **израз** със стойност
- компилаторът проверява пълнотата

switch като израз

```
static String describe(DayOfWeek day) {
    return switch (day) {
        case SATURDAY, SUNDAY -> "Почивен ден";
        case FRIDAY -> "Почти уикенд";
        default -> "Работен ден";
    };
}

static int daysIn(int month, boolean leap) {
    return switch (month) {
        case 4, 6, 9, 11 -> 30;
        case 2 -> {
            System.out.println("Февруари!");
            yield leap ? 29 : 28;    // стойност от блок
        }
        default -> 31;
    };
}
```

while и do-while

- `while` може да не се изпълни нито веднъж
- `do-while` се изпълнява поне веднъж
- Внимание с безкрайните цикли!

```java
// Сума на цифрите на число
int n = 12345, sum = 0;
while (n > 0) {
    sum += n % 10;
    n /= 10;
}
System.out.println(sum);   // 15

// Въвеждане до коректна стойност
Scanner in = new Scanner(System.in);
int age;
do {
    System.out.print("Възраст (0-120): ");
    age = in.nextInt();
} while (age < 0 || age > 120);
```

for и for-each

- `for (инициализация; условие; стъпка)`
- `for-each` – без брояч и без грешки с индекси
- В `for-each` не можем да променяме масива по индекс

```java
int[] numbers = {4, 8, 15, 16, 23, 42};

// Класически for – имаме индекс
for (int i = 0; i < numbers.length; i++) {
    System.out.println(i + ": " + numbers[i]);
}

// for-each – само стойностите
int total = 0;
for (int x : numbers) {
    total += x;
}
System.out.println("Сума: " + total);
```

break и continue с етикет

- `break` – излиза от най-вътрешния цикъл
- `continue` – прескача към следващата итерация
- С **етикет** – действат на външния цикъл

```
int[][] grid = {{1, 2, 3},
                {4, -5, 6},
                {7, 8, 9}};

search:
for (int r = 0; r < grid.length; r++) {
    for (int c = 0; c < grid[r].length; c++) {
        if (grid[r][c] < 0) {
            System.out.println(r + "," + c);
            break search;  // от двата цикъла
        }
    }
}
```

Често използвани шаблони с цикли

- **Брояч** – колко елемента отговарят на условие
- **Акумулатор** – сума, произведение
- **Максимум/минимум** – започваме от първия елемент
- **Търсене** – прекратяваме при намиране

```
int[] t = {12, -3, 7, 25, 0, 18};

int positives = 0;          // брояч
int sum = 0;                // акумулатор
int max = t[0];             // максимум
int indexOfZero = -1;       // търсене

for (int i = 0; i < t.length; i++) {
    if (t[i] > 0) positives++;
    sum += t[i];
    if (t[i] > max) max = t[i];
    if (t[i] == 0 && indexOfZero < 0) indexOfZero = i;
}
```

05

Методи и масиви

Статични методи, едномерни и многомерни масиви

Методи (функции)

```java
public class MathUtils {
    static int max(int a, int b) {
        return a > b ? a : b;
    }
    static double max(double a, double b) {     // претоварване
        return a > b ? a : b;
    }
    static void printLine(int length) {         // void — без резултат
        System.out.println("-".repeat(length));
    }

    public static void main(String[] args) {
        System.out.println(max(3, 7));          // 7
        System.out.println(max(2.5, 1.5));      // 2.5
        printLine(20);
    }
}
```

Претоварване – еднакво име, различни параметри. Параметрите се предават **по стойност**.

Рекурсия

- Метод, който извиква **сам себе си**
- Задължително: **дъно** – случай без рекурсивно извикване
- Всяко извикване заема място в стека
- Твърде дълбока рекурсия → `StackOverflowError`

```
static long factorial(int n) {
    if (n <= 1) {
        return 1;                       // дъно
    }
    return n * factorial(n - 1);        // рекурсия
}

static int gcd(int a, int b) {          // Евклид
    return b == 0 ? a : gcd(b, a % b);
}

factorial(5);   // 120
gcd(48, 18);    // 6
```

Масиви

- Масивът е **обект** с фиксирана дължина
- Индексите са от `0` до `length - 1`
- Елементите получават стойности по подразбиране (`0`, `false`, `null`)
- Излизане извън границите → `ArrayIndexOutOfBoundsException`

```java
int[] a = new int[5];          // {0, 0, 0, 0, 0}
a[0] = 10;
a[4] = 50;

double[] prices = {1.5, 2.0, 3.25};
String[] names = new String[3]; // {null, null, null}

System.out.println(a.length);    // 5
System.out.println(prices[2]);   // 3.25
// a[5] = 1;  -> ArrayIndexOutOfBoundsException
```

Класът Arrays

- `java.util.Arrays` – помощни методи
- `toString` – отпечатване
- `sort` – сортиране
- `binarySearch` – търсене в сортиран масив
- `fill`, `equals`, `copyOf`

```
int[] data = {5, 3, 9, 1, 7};
System.out.println(Arrays.toString(data));
// [5, 3, 9, 1, 7]

Arrays.sort(data);
System.out.println(Arrays.toString(data));
// [1, 3, 5, 7, 9]

int pos = Arrays.binarySearch(data, 7);
// 3

int[] ones = new int[4];
Arrays.fill(ones, 1);   // [1, 1, 1, 1]
```

Копиране на масиви

- Присвояването копира **референцията**, не данните!
- `Arrays.copyOf` и `clone()` създават нов масив
- `System.arraycopy` – копиране на част

```
int[] original = {1, 2, 3};
int[] alias = original;      // същият масив!
alias[0] = 99;
System.out.println(original[0]);   // 99

int[] copy = Arrays.copyOf(original, 3);
copy[1] = 0;
System.out.println(original[1]);   // 2

int[] bigger = Arrays.copyOf(original, 5);
// [99, 2, 3, 0, 0]

int[] dest = new int[3];
System.arraycopy(original, 0, dest, 0, 3);
```

Многомерни масиви

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(matrix[1][2]);        // 6
System.out.println(matrix.length);       // 2 реда

// "Назъбен" масив – триъгълник на Паскал
int[][] triangle = new int[5][];
for (int i = 0; i < triangle.length; i++) {
    triangle[i] = new int[i + 1];
    triangle[i][0] = triangle[i][i] = 1;
    for (int j = 1; j < i; j++) {
        triangle[i][j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
    }
}
System.out.println(Arrays.deepToString(triangle));
```

Двумерният масив е **масив от масиви** – редовете може да са с различна дължина

06

Низове и изброими типове

String, StringBuilder, text blocks, enum

Класът String

- Низът е **обект** – последователност от символи
- Низовете са **неизменяеми** (immutable)
- Всяка „промяна“ създава нов низ
- + слепва низове и други стойности

```
String s = "Програмиране";
System.out.println(s.length());           // 12
System.out.println(s.charAt(0));          // П
System.out.println(s.toUpperCase());      // ПРОГРАМИРАНЕ
System.out.println(s.substring(0, 5));    // Прогр
System.out.println(s.indexOf("рам"));     // 4

String t = s.replace("ане", "аме");
System.out.println(s);                    // непроменен!
System.out.println("Java " + 21);         // Java 21
```



Низове се сравняват с equals(), а не с ==

`==` сравнява референции (дали е един и същ обект), а `equals()` сравнява съдържанието. За сравнение без значение от регистъра – `equalsIgnoreCase()`.

Разделяне, съединяване и форматиране

- `split` – разделя по регулярен израз
- `String.join` – съединява с разделител
- `strip` – маха водещи и крайни интервали
- `formatted` / `String.format` – форматиране

```
String csv = "Иван, Мария ,Петър";
String[] parts = csv.split(",");
for (String p : parts) {
    System.out.println("[" + p.strip() + "]");
}

String joined = String.join(" | ", "a", "b", "c");
// a | b | c

String line = "%-8s%6.2f лв.".formatted("Кафе", 2.5);
// Кафе      2.50 лв.

System.out.printf("Студент: %s, оценка: %d%n", "Ана", 6);
```

Преобразуване между низове и числа

- `Integer.parseInt`, `Double.parseDouble` – от низ към число
- Невалиден вход → `NumberFormatException`
- `String.valueOf` или `+ ""` – от число към низ

```java
int n = Integer.parseInt("123");
double d = Double.parseDouble("3.14");
long big = Long.parseLong("9000000000");

String s1 = String.valueOf(42);
String s2 = Integer.toString(255, 16);   // "ff"
String s3 = Integer.toBinaryString(10);  // "1010"

try {
    int bad = Integer.parseInt("12a");
} catch (NumberFormatException e) {
    System.out.println("Невалидно число!");
}
```

StringBuilder – изменяем низ

- Слепването на `String` в цикъл е бавно
- `StringBuilder` променя един и същ буфер
- `append`, `insert`, `delete`, `reverse`

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 5; i++) {
    sb.append(i).append(", ");
}
sb.setLength(sb.length() - 2);
System.out.println(sb);   // 1, 2, 3, 4, 5

sb.insert(0, "[").append("]");
System.out.println(sb);   // [1, 2, 3, 4, 5]

String w = "капак";
boolean pal = new StringBuilder(w)
        .reverse().toString().equals(w);
```

Text blocks

- От Java 15 – многоредови низове с `"""`
- Запазват форматирането, без `\n` и `+`
- Подходящи за JSON, SQL, HTML
- Отстъпът се изчислява спрямо затварящите кавички

```
String json = """
        {
          "name": "Иван",
          "course": 2,
          "language": "Java"
        }
        """;

String sql = """
        SELECT name, grade
        FROM students
        WHERE grade >= %d
        ORDER BY name
        """.formatted(5);
```

Изброими типове (enum)

- Фиксиран набор от именувани константи
- Безопасни по тип – не може да се подаде невалидна стойност
- `values()` – всички константи, `ordinal()` – пореден номер
- `valueOf("…")` – от низ към константа

```
enum Season { WINTER, SPRING, SUMMER, AUTUMN }

Season s = Season.SUMMER;
System.out.println(s);                // SUMMER
System.out.println(s.ordinal());      // 2

for (Season x : Season.values()) {
    System.out.print(x + " ");
}

Season fromText = Season.valueOf("WINTER");
if (s == Season.SUMMER) {
    System.out.println("Ваканция!");
}
```

enum с полета, конструктор и методи

```
enum Planet {
    MERCURY(3.303e+23, 2.4397e6),
    EARTH(5.976e+24, 6.37814e6),
    JUPITER(1.9e+27, 7.1492e7);

    private static final double G = 6.67300E-11;
    private final double mass;      // kg
    private final double radius;    // m

    Planet(double mass, double radius) {
        this.mass = mass;
        this.radius = radius;
    }

    double surfaceGravity() { return G * mass / (radius * radius); }
}
```

Всеки enum е клас – може да има полета, конструктор и методи

Типични грешки на начинаещите

- `=` вместо `==` в условие
- Сравняване на низове с `==` вместо с `equals`
- Забравен `break` в класически `switch`
- Целочислено деление: `1 / 2` е `0`
- Индекс `arr[arr.length]` – извън масива
- Точка и запетая след `if (...)` или `for (...)` – празно тяло!
- Грешна главна/малка буква: `string`, `system.out`

Обобщение

- Java се компилира до **байткод**, който JVM изпълнява на всяка платформа
- Използваме съвременна LTS версия – **Java 21** (и новостите на Java 25)
- 8 примитивни типа; всичко останало е **референтен** тип
- `var` извежда типа, но той остава статичен
- `switch` със стрелка може да бъде **израз**
- Масивите са обекти с фиксирана дължина; `Arrays` помага
- `String` е неизменяем – сравняваме с `equals()`; за много промени – `StringBuilder`
- `enum` – типово безопасни константи с поведение

Въпроси за самопроверка

- Каква е разликата между JDK и JVM?
- Защо Java програмите са платформено независими?
- Какво ще отпечата `System.out.println(5 / 2 + 5 % 2)`?
- Кога `var` не може да се използва?
- С какво switch изразът е по-добър от класическия switch?
- Защо `s1 == s2` може да върне `false` за два еднакви низа?
- Кога използваме `StringBuilder` вместо `String`?

Въпроси?

Следваща лекция: Класове и обекти

